

수동 Release 0줄로 상주 메모리 회수 — 측정으로 증명

Addressables 위에 수명 스코프를 얹어, "안전한 쪽=Global"로 도피하던 리소스를 회수 가능한 수명으로 재배치했다.

-60%

진입 Global 메모리
513MB → 205MB

-41%

진입 전체 합계
617MB → 363MB

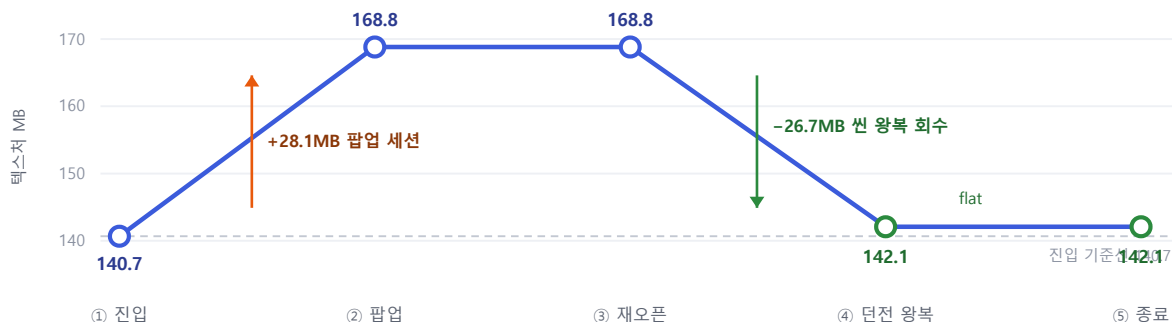
-72%

세션 단조 증가
+56MB → +16MB

flat

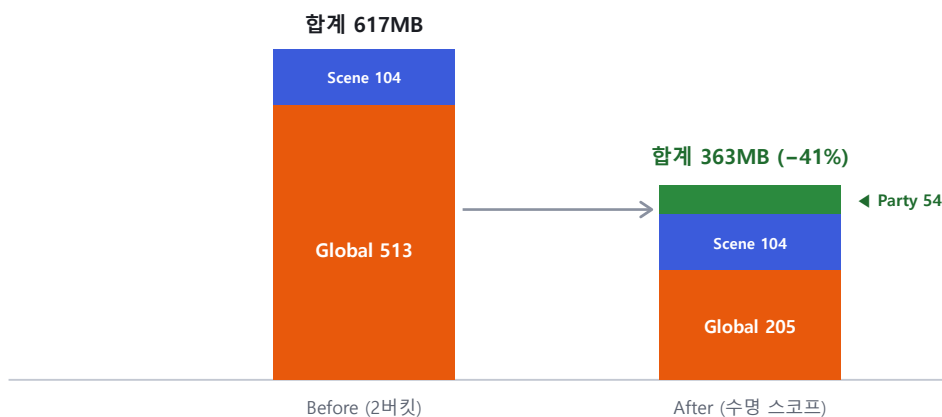
왕복 후 누수
358.6 → 359.1MB

① 텍스처 메모리 — 씬 왕복에서 자동 회수 개발 빌드 실측



팝업이 Scene 스코프에 적재됐다가(②) 씬 전환에서 refCount 자동 정산으로 반납(④, 진입 수준 복귀). 재오픈(③)은 캐시 히트라 무증가. 수동 Release 호출 0줄.

② 진입 직후 메모리 구조 — Global 독점 → 수명별 분산



Global(영구) ■ Scene(씬) ■ Party(파티, 신설)

Global 덩어리(513)가 205로 줄고, 나머지는 회수 가능한 Scene-Party 수명으로 분산. 감축분 대부분은 비파티 캐릭터 8종이 아예 로드되지 않게 된 것(라벨 다이어트 42→12개).

추가 증거 (같은 계측)

주장	측정 근거	수치
재오픈 = 무비용	같은 팝업 첫 오픈 vs 재오픈 요청→표시 ([UIMetric])	UI_Info 23.1ms → 2.3ms
던전 왕복 영구 증가 해소	왕복 전후 Global 영구 증가 (Baseline v2 → 완료)	+18.7MB → ≈0
풀↔에셋 수명 안전	refCount 티켓 왕복 (Global/Scene/Party 전 버킷)	358.6 → 359.1MB (flat)
고아·누수 0	[?] 버킷 · [ResourceLeak] · [PoolLeak] 카운트	전부 0

트레이드오프 (정직하게 동반)

진입 -308MB 상시 부담을 팝업 첫 오픈 +14~20ms 일회성 지연으로 바꾼 거래. 재오픈은 회귀 없음(~9ms). 던전 스크립트·프레임 예산 스트리밍은 측정 근거로 제외(던전 왕복이 이미 +0.9MB) = 필요한 것만 했다는 증거.

신뢰도 · 방법론

Addressables Use Existing Build(실제 번들) → 증감·비율 신뢰도 "강" · 절대값은 개발 빌드 실측으로 확정(에디터 x2 사본 없음). 측정 도구의 x6.8 계통 오차를 교차 검증으로 자가 발견·보정. 히치는 n=1~3라 15ms 이하 차이는 주장하지 않음.